

DevOps Task Manager

Production-Ready CI/CD Pipeline & AWS
Infrastructure Management

GitHub Repository:

github.com/Komalpr3t/devops-task-manager

Report Type: Project Specification & Deployment Blueprint

Date: May 2026

Project Report: DevOps Task Manager

Production-Ready Remote CI/CD Pipeline & AWS Infrastructure Management

GitHub Repository

- **Repository Link:** <https://github.com/Komalpr3t/devops-task-manager>
-

1. Executive Summary

This report outlines the deployment architecture, configuration steps, and automated CI/CD pipeline for the **DevOps Task Manager** application. The project is designed to prioritize stability, resource efficiency, and simplicity:

- **Phase 1 — One-Time Server Setup:** Terraform provisions the AWS EC2 instance (`t3.micro` default) with Docker and `k3s` automatically installed via `user_data` (optimized with `--disable traefik --disable servicelb` to conserve memory).
- **Phase 2 — Automated CI/CD:** An automated Jenkins pipeline running locally on the developer's Windows machine checks out the source code, builds the Docker image, pushes it to Docker Hub, and executes a remote deployment via SSH.
- **Monitoring Architecture:** To operate stably within AWS free-tier limit constraints (1GB RAM on `t3.micro`), we adopt a hybrid observability model. Prometheus and Grafana run locally on the developer's machine using Docker Compose. Local Prometheus runs on port 9091 and Grafana on port 3000, scraping metrics remotely from lightweight exporters (`node_exporter` and `kube-state-metrics`) running on the EC2 host.

By keeping the heavy build engine (Jenkins), query database (Prometheus), and visualization UI (Grafana) running locally, we keep the remote EC2 instance's memory footprint extremely low and guarantee high cluster stability.

2. Phase 1 — One-Time Server Setup

2.1 Host Specifications & Sizing Strategy

Running container orchestration and monitoring stacks on limited cloud resources requires careful memory budgeting. * **Instance Type Selection:** `t3.micro` (2 vCPUs, 1GB RAM) * **Sizing Strategy:** To operate within AWS free-tier limit constraints, we deploy on a single `t3.micro` instance. * **Optimization Strategy:** Running heavy Docker builds and Jenkins pipeline orchestration locally saves critical CPU and memory on the target server. Furthermore, the remote `k3s` cluster is optimized by disabling Traefik and ServiceLB, and the database metrics storage is offloaded to the local machine, preserving the EC2 instance's RAM for the core `task-manager` application.

2.2 Security Group Rules

The security group is configured to allow administration, API access, application routing, and remote metric collection:

Port	Protocol	Purpose	Access Source
22	TCP	SSH — Remote administration and Jenkins pipeline deployment	Restricted to Developer/Jenkins IP
80	TCP	HTTP — Task Manager application routing	Any (<code>0.0.0.0/0</code>)
6443	TCP	Kubernetes API — Remote <code>kubect1</code>	Restricted to Admin IP
9100	TCP	Node Exporter — Host system metrics	Restricted to Admin/Prometheus IP
30080	TCP	App NodePort — Exposes Task Manager frontend	Any (<code>0.0.0.0/0</code>)
30091	TCP	Kube-State-Metrics NodePort — Exposed cluster state metrics	Restricted to Admin/Prometheus IP

2.3 Auto-Installation via User Data

Terraform automatically bootstraps the instance using the `user_data` script, which installs Docker and an optimized `k3s` cluster:

```
# Install Docker Engine
apt-get update -y
apt-get install -y docker-ce docker-ce-cli containerd.io
usermod -aG docker ubuntu

# Install optimized k3s (excluding Traefik & ServiceLB to save 500MB+ RAM)
curl -sL https://get.k3s.io | sh -s - \
  --write-kubeconfig-mode 644 \
  --node-external-ip=$(curl -s http://169.254.169.254/latest/meta-data/public-ipv4) \
  --disable traefik \
  --disable servicelb
```

2.4 Remote Exporters Setup (One-Time)

SSH into the EC2 instance and set up the lightweight exporters: 1. **Setup Node Exporter (Host System Metrics)** Create a standard systemd service to run Node Exporter continuously on port 9100: ``ini [Unit] Description=Node Exporter After=network.target

[Service] User=ubuntu ExecStart=/usr/local/bin/node_exporter

[Install] WantedBy=multi-user.target `` 2. **Setup Kube-State-Metrics (Cluster State Metrics)**** The manifest file (`k8s/kube-state-metrics.yaml`) is copied and applied dynamically as part of the Kubernetes directory rollout. It configures the read-only cluster roles for resource state discovery and exposes metrics on NodePort 30091`.

3. Phase 2 — Automated CI/CD

3.1 Repository Structure

```
devops-task-manager/  
├─ app/                # React frontend source code (Vite + React)  
├─ docker/             # Dockerfile for multi-stage image build  
├─ k8s/                # Kubernetes Deployment and Service manifests  
│   ├── deployment.yaml  
│   ├── service.yaml  
│   └─ kube-state-metrics.yaml  
├─ terraform/          # Terraform configurations  
│   ├── main.tf  
│   ├── variables.tf  
│   ├── outputs.tf  
│   └─ terraform.tfvars  
├─ jenkins/            # Jenkinsfile pipeline definition  
└─ monitoring/          # Local observability configurations  
    ├── prometheus-local.yml  
    └─ docker-compose.yml
```

3.2 Automated CI/CD Workflow

The local Jenkins server coordinates the delivery pipeline: 1. **Developer pushes code** to GitHub repository. 2. **Local Jenkins server** pulls the repository updates. 3. **Jenkins builds the React/Vite application** and builds a multi-stage Docker image. 4. **Jenkins pushes** the dual-tagged image (`latest` and `build-${BUILD_NUMBER}`) to Docker Hub. 5. **Jenkins queries AWS EC2 API** via PowerShell to fetch the latest public IP of the target node. 6. **Jenkins copies Kubernetes manifests** over to the EC2 host via SCP. 7. **Jenkins executes remote deployment commands** via SSH, pointing to the k3s configuration (`export KUBECONFIG=/etc/rancher/k3s/k3s.yaml`) and applying the manifests. 8. **k3s pulls the latest image** from Docker Hub, updates the pods, and Jenkins verifies the rollout status.

4. Implementation Highlights

4.1 Dockerfile Highlight (docker/Dockerfile)

The React app is built using a multi-stage compilation flow to keep the runtime image minimal and secure:

```
# Stage 1: Build static assets
FROM node:20-alpine as build
WORKDIR /app
COPY app/package.json app/package-lock.json* ./
RUN npm install
COPY app/ ./
RUN npm run build

# Stage 2: Serve using Nginx
FROM nginx:alpine
RUN rm -rf /usr/share/nginx/html/*
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

4.2 Jenkinsfile Flow (jenkins/Jenkinsfile)

Rather than dumping the entire file, the key lifecycle stages of our deployment are highlighted below:

```

pipeline {
  agent any
  environment {
    DOCKER_IMAGE = 'komalpreet1703/task-manager'
    PEM_FILE = 'C:\\jenkins-keys\\task-manager.pem'
    // Windows tool overrides for path compatibility
    GIT_SSH = 'C:\\Program Files\\Git\\usr\\bin\\ssh.exe'
    GIT_SCP = 'C:\\Program Files\\Git\\usr\\bin\\scp.exe'
  }
  stages {
    stage('Build & Push Docker Image') {
      steps {
        bat 'docker build -t %DOCKER_IMAGE%:latest -f docker/Dockerfile .'
        bat 'docker push %DOCKER_IMAGE%:latest'
      }
    }
    stage('Deploy to Kubernetes') {
      steps {
        script {
          // Fetch EC2 IP dynamically via AWS CLI CLI
          def EC2_IP = powershell(script: 'aws ec2 describe-instances ... --q

          // Copy manifests
          bat '"%GIT_SCP%" -o StrictHostKeyChecking=no -i "%PEM_FILE%" -r k8s

          // Remotely apply and verify deployment status
          bat '"%GIT_SSH%" -o StrictHostKeyChecking=no -i "%PEM_FILE%" ubuntu@
        }
      }
    }
  }
}

```

4.3 Kubernetes Deployment Highlights (k8s/deployment.yaml)

The deployment is tuned to enforce strict resource limitations, ensuring cluster stability under minimal sizing:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: task-manager
spec:
  replicas: 2
  template:
    spec:
      containers:
      - name: task-manager
        image: komalpreet1703/task-manager:latest
        imagePullPolicy: Always
        resources:
          requests:
            memory: "64Mi"
            cpu: "50m"
          limits:
            memory: "128Mi"
            cpu: "100m"
```

5. Observability & Monitoring Setup

5.1 Local Prometheus Scrape Configuration (monitoring/prometheus-local.yml)

Local Prometheus runs via Docker Compose on the developer's machine on port 9091 and is configured to scrape the remote metrics endpoints directly over the internet:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'remote-ec2-node'
    static_configs:
      - targets: ['44.195.19.206:9100'] # Remote host node-exporter

  - job_name: 'remote-kube-state-metrics'
    static_configs:
      - targets: ['44.195.19.206:30091'] # Remote cluster state metrics
```


5.2 Local Observability Compose Configuration (monitoring/docker-compose.yml)

A simple, robust container definition manages the monitoring services locally:

```
version: '3.8'
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: local-prometheus
    volumes:
      - ./prometheus-local.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9091:9090"
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: local-grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin123
    restart: unless-stopped
```

6. Practical Windows Jenkins Deployment Challenges

Running a remote Linux-targeting pipeline from a Windows Jenkins environment presents several unique engineering challenges: 1. **Path Resolution & Command Inheritance:** Windows shell commands executed under Jenkins do not inherit the user-level path variable configuration. To bypass OpenSSH path execution errors and ensure commands use the correct tools, we explicitly declare variables pointing to the Git Bash bundle binaries (`GIT_SSH` and `GIT_SCP`). 2. **Private Key Permissions (`task-manager.pem`):** In Windows environments, standard ssh/scp binaries often throw file permission errors if the SSH key permissions are too permissive. By storing the private key locally at a secure location (`C:\jenkins-keys\task-manager.pem`) and referencing it directly, the Windows binaries load the key file securely. 3. **Multi-line Command Continuations:** The Windows command processor (`cmd.exe`) uses the caret symbol (`^`) as the

multi-line continuation character rather than the Unix backslash (\). Triple-quoted `bat` steps inside the pipeline are formatted with the caret to ensure correct execution.

7. Conclusion

Separating the DevOps workflow into a one-time server bootstrap phase and a dynamic local CI/CD deployment phase maximizes deployment stability. By offloading metrics collection (Prometheus) and dashboarding (Grafana) to the local developer environment, the remote `t3.micro` EC2 instance stays lightweight and operates efficiently within free-tier resource limits. This hybrid architecture simulates a professional, production-ready DevOps infrastructure pipeline.